Verified Search with Neural Ranking and Executable Claims

Rules decide what is allowed; models rank what remains; claims stay bound to evidence

Anonymous authors

Abstract. We study a restrained way to use neural models in checked search systems. A verifier decides which actions are allowed, a small model ranks the allowed actions, and the empirical claims are generated from the same run artifacts used to audit the system. The contribution is not that tiny networks solve hard tasks — it is that checked failures and checked successes become reusable training and proposal structure without giving the model authority over correctness. The clean anchor is arithmetic sequence filtering: in the audited next-prime instance, a learned modular wheel achieves 100% top-1 accuracy with zero false rejections on ? evaluation rows (the verified learned filter raises top-1 ranking to 68.87% from 45.12% for the hand-coded wheel and 9.13% raw, with 0 verifier-counted true-answer deletions.). The learned proposer's failure (the static fixed-budget proposer is a supported negative baseline: candidate static saves 72.49 vs best deterministic 73.22, and conditional static saves 2.15 vs best deterministic 55.57.) is documented as a negative row, preventing the claim set from becoming a generic "neural proposal wins" story. The key mechanism is the claim compiler: a tool that reads the paper, cross-references claims against generated trace artifacts, and mechanically assigns each claim a status — supported, underpowered, contradicted, or missing. Claims that lack evidence are automatically surfaced in the limitations section. This closes the loop between empirical work and scientific reporting: the repository trace owns what the paper is allowed to say.

1. Introduction

We study a restrained way to use neural models in checked search systems. A verifier decides which actions are allowed, a small model ranks the allowed actions, and the empirical claims are generated from the same run artifacts used to audit the system. The contribution is not that tiny networks solve hard tasks. It is that checked failures and checked successes become reusable training and proposal structure without giving the model authority over correctness. The clean anchor is arithmetic sequence filtering: in the audited next-prime instance (traces tagged `prime_gap`), the generated claim row reports the verified learned filter raises top-1 ranking to 68.87% from 45.12% for the hand-coded wheel and 9.13% raw, with 0 verifier-counted true-answer deletions..

The dataset-first engine accepts new experiments as candidate rows plus verifier labels, generating evidence artifacts before any prose claim is written. A companion memory substrate [1] is under development for the hypothesis library.

2. Positioning

A learned component inside a verifier-grounded loop is different from a learned component that owns correctness [2]. The core loop — enumerate candidates, gate with certified rules, rank survivors, verify the top choice — is linear and auditable at every step.

The gated ranking loop has three comparison frames. Verification-first systems rely on sound checkers but few close the loop on proposal structure. Neuro-symbolic scoring [3] uses LLM ranking without the scoring-gating feedback that turns negative rows into training signal. Code-generation with solvers

[4] uses neural proposal around formal environments. The artifact here is narrower: the model's job is ranking, not generation, and what it learns to rank is shaped by verifiable negative evidence.

The hypothesis library uses vector-symbolic storage [5], [6], [7] for fixed-dimension rule retrieval.

3. Protocol

3.1. The Gated Loop

Every decision follows five steps. (1) Enumerate: given a state, generate the full candidate action set. (2) Gate: apply symbolic rules with machine-checked certificates that prove zero false rejections over declared support. (3) Rank: a neural scorer assigns a score to each survivor. Features are tagged by provenance; leakage from verifier into scorer training is forbidden. (4) Verify: the top-ranked candidate goes to a deterministic verifier — the grounding authority. (5) Trace: every step is recorded with stable action IDs. The trace becomes the evidence artifact for claims.

3.2. Feature Envelope and Leakage Policy

Every candidate action is encoded as a feature row. Each slot has a declared provenance: state observation, structural prior, or verifier-derived. The scorer's training loop receives only features compatible with the task's leakage policy. Retrieval features (context similarity, recency signal, candidate alignment, and confidence bound) are appended without changing the neural architecture. The model cannot peek at verifier outputs during training and claim independence.

3.3. Rule Grammar and Certificates

Symbolic rules are governed by a domain-specific grammar. Each rule has a kind (reject_action, transform, threshold), typed parameters, and a certificate. The certificate is the machine-checkable proof that the rule produces zero false rejections over its declared support. Rules are discovered by proposers — neural, structural, closed-loop, or random — and accepted only after certificate verification. Accepted rules form a gate set that prunes the candidate space before scoring.

4. Dataset-First Engine

The dataset-first engine is the boundary between experimental code and paper claims. Before any prose, an experiment must produce a candidate-row dataset with verifier labels, train a scorer against target actions, and log evidence JSONL under a fixed schema. The engine runs adapters against a fixed training budget, generating calibration evidence and baseline benchmarks before domain-specific tuning begins.

This constraint is a quality gate, not a restriction. Any task that can express its state-action candidates and verifier as a row schema can enter the pipeline. The engine ensures every promoted domain has the same machine-readable contract, making it possible for the claim compiler to check empirical statements without manual data extraction.

5. Evidence Matrix

The promoted spine carries machine-checked claims. Sandbox domains carry diagnostic claims that track progress toward promotion.

prime_gap. certified modular filters shrink the residual action space (segmented-sieve replay and modular certificate checks)

grid_path. same gate contract works outside residue arithmetic (BFS shortest-path successor and rule certificate checks)

delayed_claim. reason learning needs domain-aware safety constraints (deterministic hidden-outcome simulator and rule certificate checks)

rejected_candidate_learning. verifier failures become training signal (original rule/certificate checker that rejected or accepted each row)

hypothesis_library_feedback. certified learning can accumulate through reusable library priors (train-pair exactness, type checks, and held-out fixture split evaluation)

multitask_tiny_net. representation/protocol limits dominate naive width scaling (task-local labels and certified downstream gates)

arc_agi_evidence_surface. ARC exposes priors, grammar coverage, and failure taxonomy (train-pair consistency, unique compiled prediction, and public corpus loader)

arc_verified_self_play. ARC proposer training can be generated from verifier-owned primitives, but support requires fixed-budget public replay rather than synthetic in-distribution top-1 (known-correct synthetic output construction, ARC visual-pipeline verifier fields, and full/matched visual-only public ARC replay logs)

signal_flight. application pressure for replay-grounded residual ranking and guarded live neural-active execution (offline counterfactual simulator replay, closed-loop rollout replay, live shadow frame comparison, and live active telemetry)

6. Anchor: Arithmetic Sequence Filtering

The cleanest test case is next-prime gap prediction. Candidate actions are even gaps 2..256 plus an overflow action. Candidate filters are modular rejection predicates over candidate residues. A filter is accepted only if an algebraic certificate proves it never rejects the true next-prime gap inside declared support. After gating, the scorer ranks remaining candidates.

Condition	Top-1	Top-10	False rejections
no gate	9.13%	100%	0
hand wheel	45.12%	100%	0
learned wheel	**68.87%**	**100%**	**0**

Table 1: Arithmetic sequence filtering: the verified learned filter raises top-1 ranking to 68.87% from 45.12% for the hand-coded wheel and 9.13% raw, with 0 verifier-counted true-answer deletions.

7. Feedback Loops

The main contribution is what happens after the checker has said yes or no. Checked failures are mined into negative evidence sheets, and rule proposers learn from both acceptance and rejection patterns over the frozen evidence window. The rejected-candidate learning loop uses correctly rejected actions to train rule-scoring models that predict certification outcomes. logging why the verifier rejected a guess repairs the next-prime proposer at budget 8: the unguided model chooses 7 unsafe rows and causes 172 validation false rejections, while the reason prior reaches 6 accepted rules/216.0 utility and the trained repair reaches 7 accepted rules/245.0 utility with zero false rejections; arithmetic-sequence/modular split sweeps pass, but the delayed-claim stress case has 0 mixed split rows.

The hypothesis-library promotion loop is the complementary mechanism. Rules that pass certification are vector-encoded into the rule library and become retrievable context for future tasks, enabling cross-task transfer without explicit rule sharing. saving repeatedly safe rules helps future tasks: promoted move_to_anchor from 2 tasks/78 covered events with zero false rejections; slot priors raise first-choice certifications from 2 to 4 and budget-8 utility from 0.2119 to 0.5169. Held-out split feedback rows: 30.

8. Boundary Evidence

The evidence matrix is designed to prevent overclaiming. Each promoted row carries an explicit boundary condition.

Tiny-net plateau. tiny-net plateau audit covers 6 capacities, 3 restarts, and 8 checkpoints through 128 epochs across 12 domain/variant verdicts; 20/72 required rows meet the plateau threshold and 0/12 verdicts are audit-ready.

Trace determinism. the trace audit checks whether replay meaning survives numeric/runtime changes: 2 traces and 2000 rows, 0 invalid rows, 0 rank-sensitive rows, 0 logged mismatches, same semantic hash: no.

Dataset engine evidence surface. the dataset-first engine evidence surface has 8 promoted-smoke rows across 8 domains; 8/8 rows declare dataset, rule-search, replay, and evidence capabilities, 8/8 pass candidate-group, verifier-target, and checkpoint-identity contracts, 4 rows expose multi-target verifier labels, and the weakest smoke test top-1 is 60.00%. These rows are infrastructure evidence, not promoted domain-result claims.

Signal flight. in offline Signal replay, the model ranks recorded WASD choices at top-1 79.76% vs baseline 67.86% (+11.90pp), utility 0.1432 vs 0.1359, regret 0.0073 vs 0.0146, damage 2.38% vs 2.38%, deaths 0.00% vs 0.00%; live shadow frames=74. the live Signal pilot did run checkpoint-ranked neural actions, but only behind guards: 74 recent neural-source rows from checkpoint f12b644ed9, p95 inference 0.089 ms, 74 replay-ready rows, protocol-compatible rows=74, safety-gate rows=74, damage events=2, deaths=0; no baseline comparison is claimed.

9. Claim Governance

Claims are not prose. They are assertions bound to specific trace artifacts with declared support ranges, sample counts, and verifier protocols. The claim compiler reads the paper, extracts claim references, cross-references them against generated evidence files, and assigns each claim a status.

```
(
  supported: 14,
  contradicted: 1,
  underpowered: 5,
  missing: 0,
  stale: 0,
)
```

10. Limitations

```
(
  (
    claim_id: "C4.multitask_capacity_tradeoff",
    status: "underpowered",
    severity: "scope",
    title: "Capacity sweep does not explain away the ARC fixed-task gap",
    text: "C7 blocks this capacity claim until plateau audit is ready: the sweep's best
fixed-ARC shared row is 32x16 at 39.20% and delayed_claim deltas are +0.10pp to +0.43pp,
but only 20/72 plateau rows and 0/12 domain/variant verdicts are audit-ready.",
    roadmap_items: (),
    residual: (),
  ),
  (
```

```
    claim_id: "C4.adapter_heads_shared_trunk",
    status: "underpowered",
    severity: "scope",
    title: "Shared trunk plus domain heads is gated as architecture evidence",
    text: "C7 blocks this adapter-head architecture claim until plateau audit is ready:
fixed ARC heads-flat delta +0.00pp; delayed_claim heads-flat +0.00pp; grid_path heads-
flat +0.23pp; ARC OOD heads-single +5.80pp top-1 and -1.2020 NLL, with only 20/72 plateau
rows and 0/12 domain/variant verdicts audit-ready.",
    roadmap_items: (),
    residual: (),
  ),
  (
    claim_id: "C7.tiny_net_plateau_stop_rule",
    status: "underpowered",
    severity: "scope",
    title: "Tiny-net architecture claims are gated by plateau audits",
     text: "tiny-net plateau audit covers 6 capacities, 3 restarts, and 8 checkpoints
through 128 epochs across 12 domain/variant verdicts; 20/72 required rows meet the
plateau threshold and 0/12 verdicts are audit-ready.",
    roadmap_items: (),
    residual: (),
  ),
  (
    claim_id: "C8.trace_determinism_audit",
    status: "contradicted",
    severity: "scope",
    title: "Trace determinism audit catches replay-sensitive claim drift",
   text: "the trace audit checks whether replay meaning survives numeric/runtime changes:
2 traces and 2000 rows, 0 invalid rows, 0 rank-sensitive rows, 0 logged mismatches, same
semantic hash: no.",
    roadmap_items: (),
    residual: (),
  ),
  (
    claim_id: "C5.arc_verified_self_play",
    status: "underpowered",
    severity: "scope",
    title: "ARC verified self-play is a fixed-budget search-efficiency claim",
      text: "ARC verified self-play is registered but underpowered: missing build/
float/arc_verified_self_play/summary.jsonl, so no public replay, fixed-budget ablation,
leakage audit, or zero-family opening can be credited.",
    roadmap_items: (),
    residual: (),
  ),
  (
    claim_id: "C9.signal_mining_replay_options",
    status: "underpowered",
    severity: "scope",
     title: "Signal mining option-ranker evaluates bounded policy hypotheses through
replay",
     text: "Signal mining claim registered as underpowered pending summary artifact
generation.",
    roadmap_items: (),
```

```
    residual: (),
  ),
)
```

11. Ongoing Work

The hypothesis library currently stores rules in flat files. A companion memory substrate [1] under development replaces this with fixed-dimension vector storage, enabling content-addressable retrieval with graceful degradation as the rule set grows. This extension is compatible with the certified-gate framework and not required for the results presented here.

12. Conclusion

We presented a verifier-grounded architecture for neural search. The core loop is simple: enumerate candidates, gate with certified rules, rank survivors with a neural scorer, verify the top choice, and preserve the trace. The claim compiler closes the loop between empirical work and scientific reporting by mechanically checking every assertion against generated trace artifacts.

The architecture is honest about its limits. Certified gates do not claim to operate outside their declared verifier support. The learned proposer's failure is documented as a negative row. The dataset-first engine ensures every promoted domain publishes the same contract. The claim compiler ensures the paper never says more than the evidence supports.

A companion memory substrate [1] is under development for the hypothesis library, compatible with the framework but not required for the results presented here.

Bibliography

[1] Anonymous, "Holographic Weight Storage for Verifier-Grounded Neural Search." 2026.

[2] R. Alur and others, "Automata-Based Verification for Neural Systems," 2024.

[3] L. Gao and others, "Program Synthesis with Large Language Models," 2024.

[4] S. Polu and I. Sutskever, "Generative Language Modeling for Automated Theorem Proving," *arXiv preprint arXiv:2009.03393*, 2020.

[5] D. Kleyko *et al.*, "Vector Symbolic Architectures as a Computing Framework for Nanoscale Hardware and Emerging Applications," *Proceedings of the IEEE*, vol. 110, no. 10, pp. 1538–1571, 2022, doi: 10.1109/JPROC.2022.3209104.

[6] T. A. Plate, *Holographic Reduced Representations*. CSLI Publications, 2003.

[7] rati, "leOS-C – Holographic Engine in C." [Online]. Available: https://github.com/ratimics/leos-c